

2025년 상반기 K-디지털 트레이닝

스트림 요소 처리

[KB] IT's Your Life



1 스트림이란?

✓ 스트림

- Java 8부터 컬렉션 및 배열의 요소를 반복 처리하기 위해 스트림 사용
- 요소들이 하나씩 흘러가면서 처리된다는 의미

```
Stream<String> stream = list.stream();  
stream.forEach( item -> //item 처리 );
```

- List 컬렉션의 stream() 메소드로 Stream 객체를 얻고, forEach() 메소드로 요소를 어떻게 처리할지를 람다식으로 제공
- 스트림과 Iterator 차이점
 - 1) 내부 반복자이므로 처리 속도가 빠르고 병렬 처리에 효율적
 - 2) 람다식으로 다양한 요소 처리를 정의
 - 3) 중간 처리와 최종 처리를 수행하도록 파이프 라인을 형성

 StreamExample.java

```
package ch17.sec01;

import java.util.HashSet;
import java.util.Iterator;
import java.util.Set;
import java.util.stream.Stream;

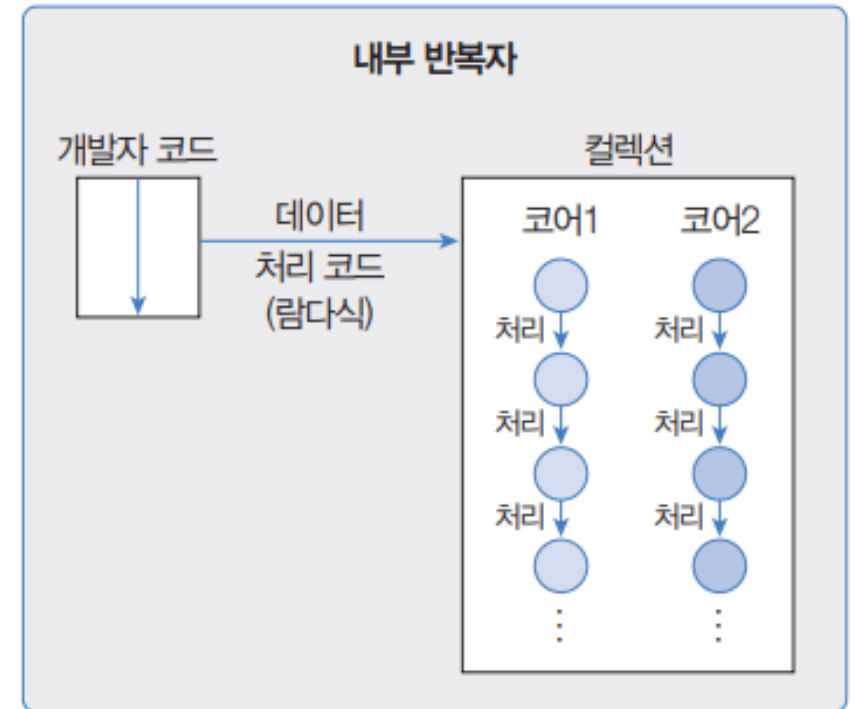
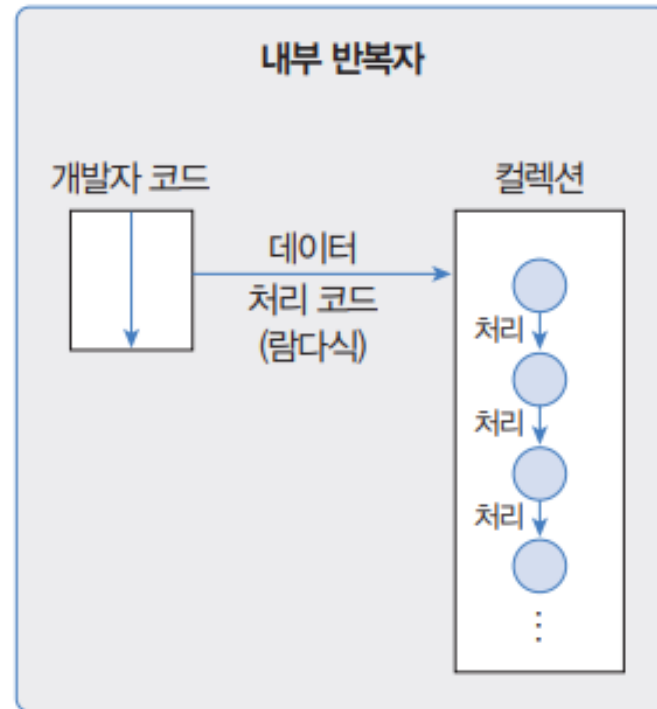
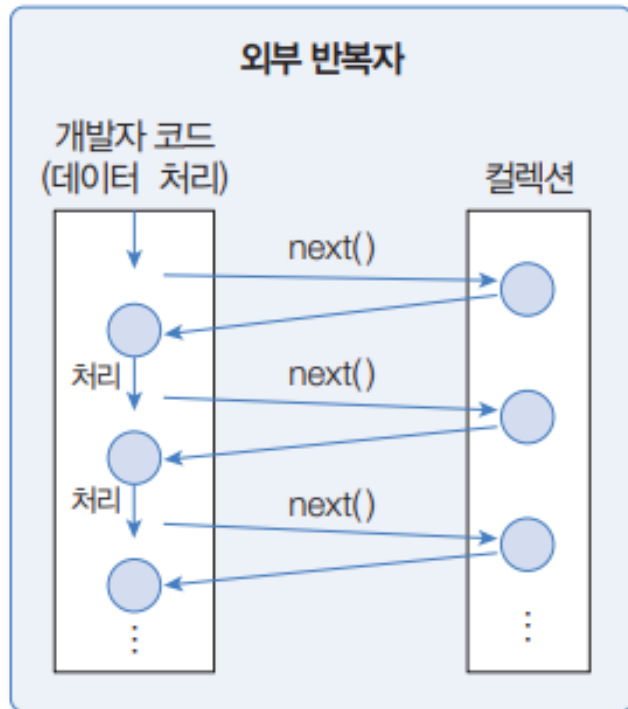
public class StreamExample {
    public static void main(String[] args) {
        //Set 컬렉션 생성
        Set<String> set = new HashSet<>();
        set.add("홍길동");
        set.add("신용권");
        set.add("감자바");

        //Stream을 이용한 요소 반복 처리
        Stream<String> stream = set.stream();
        stream.forEach( name -> System.out.println(name) );
    }
}
```

홍길동
신용권
감자바

☑ 내부 반복자

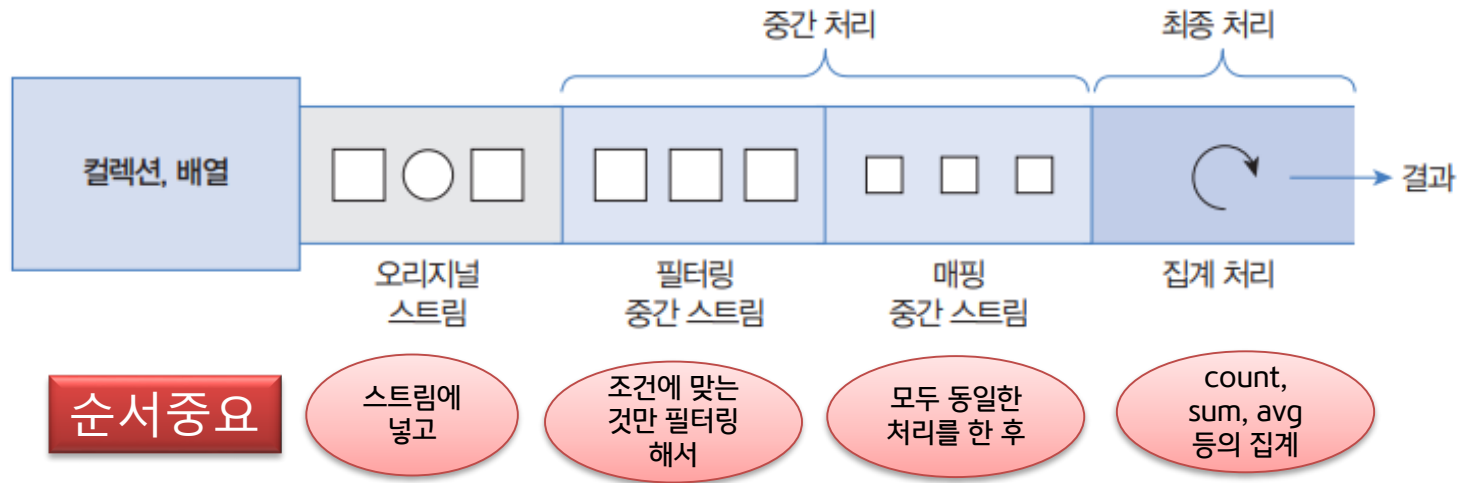
- 요소 처리 방법을 컬렉션 내부로 주입시켜서 요소를 반복 처리
- 개발자 코드에서 제공한 데이터 처리 코드(람다식)를 가지고 컬렉션 내부에서 요소를 반복 처리
- 내부 반복자는 멀티 코어 CPU를 최대한 활용하기 위해 요소들을 분배시켜 병렬 작업 가능



3 중간 처리와 최종 처리

✓ 스트림 파이프라인

- 컬렉션의 오리지널 스트림 뒤에 필터링 중간 스트림이 연결될 수 있고, 그 뒤에 매핑 중간 스트림이 연결될 수 있음

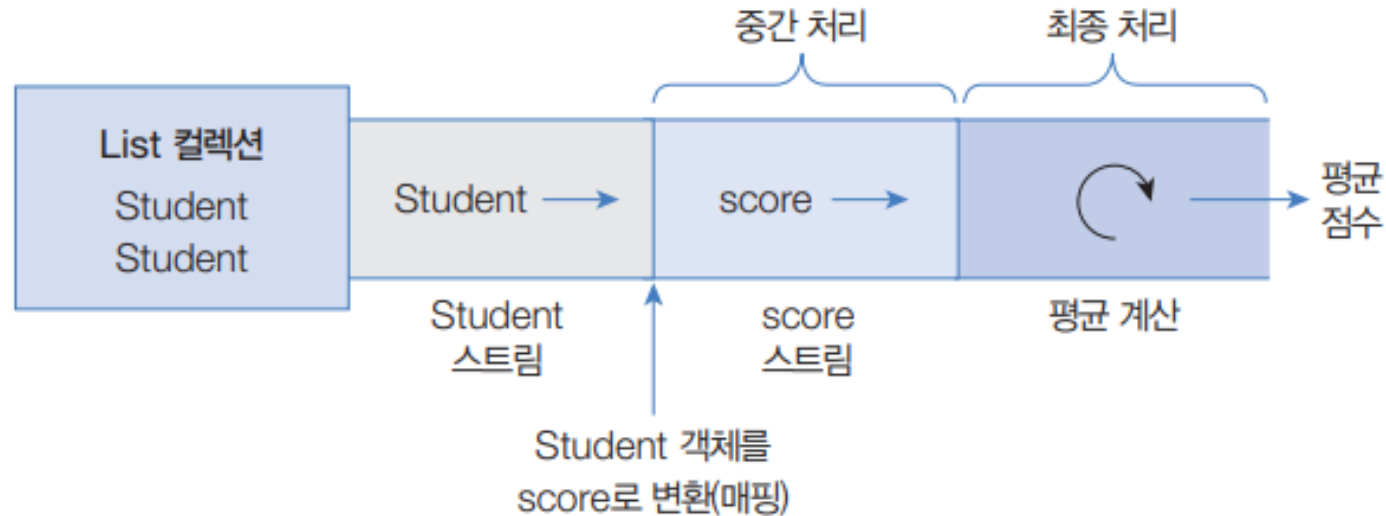


- 일단 스트림으로 만들어야함. 그 다음에 필터링도 하고, 매핑도 하고, 집계도 할 수 있음.
- 오리지널 스트림과 집계 처리 사이의 중간 스트림들은 최종 처리를 위해 요소를 걸러내거나(필터링), 요소를 변환시키거나(매핑), 정렬하는 작업을 수행

3 중간 처리와 최종 처리

✓ 스트림 파이프라인

- 최종 처리는 중간 처리에서 정제된 요소들을 반복하거나, 집계(카운팅, 총합, 평균) 작업을 수행



```
//Student 스트림
Stream<Student> studentStream = list.stream();
//score 스트림
IntStream scoreStream = studentStream.mapToInt( student -> student.getScore() );
//평균 계산
double avg = scoreStream.average().getAsDouble();
```

Student 객체를 getScore() 메소드의 리턴값으로 매핑

3 중간 처리와 최종 처리

✓ 스트림 파이프라인

○ 메소드 체이닝 패턴

```
double avg = list.stream()  
    .mapToInt(student -> student.getScore())  
    .average()  
    .getAsDouble();
```



3 중간 처리와 최종 처리

Student.java

```
package ch17.sec03;

public class Student {
    private String name;
    private int score;

    public Student (String name, int score) {
        this.name = name;
        this.score = score;
    }

    public String getName() {
        return name;
    }
    public int getScore() {
        return score;
    }
}
```


3 중간 처리와 최종 처리

StreamPipeLineExample.java

```
package ch17.sec03;

import java.util.Arrays;
import java.util.List;

public class StreamPipeLineExample {
    public static void main(String[] args) {
        List<Student> list = Arrays.asList(
            new Student("홍길동", 10),
            new Student("신용권", 20),
            new Student("유미선", 30)
        );

        //방법1
        Stream<Student> studentStream = list.stream();
        //중간 처리(학생 객체를 점수로 매핑)
        IntStream scoreStream = studentStream.mapToInt(student -> student.getScore());
        //최종 처리(평균 점수)
        double avg = scoreStream.average().getAsDouble();
        System.out.println("평균 점수: " + avg);
    }
}
```

평균 점수: 20.0

3 중간 처리와 최종 처리

StreamPipeLineExample.java

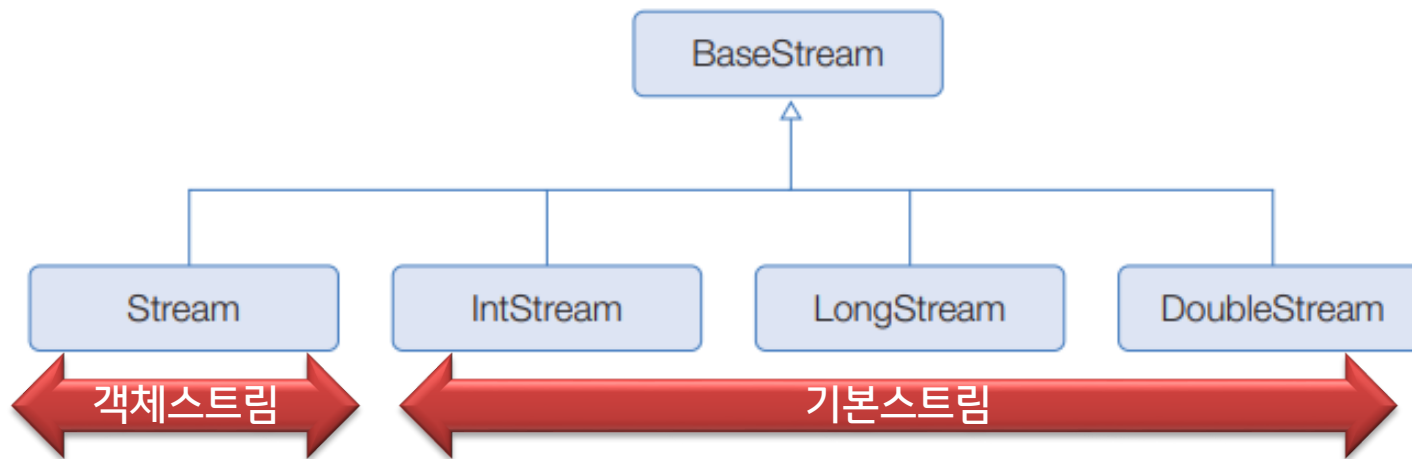
```
//방법2
double avg = list.stream()
    .mapToInt(student -> student.getScore())
    .average()
    .getAsDouble();

System.out.println("평균 점수: " + avg);
}
```

평균 점수: 20.0

✓ 스트림 인터페이스

- java.util.stream 패키지에는 BaseStream 인터페이스를 부모로 한 자식 인터페이스들은 상속 관계
 - BaseStream에는 모든 스트림에서 사용할 수 있는 공통 메소드들이 정의
 - 기본 스트림: int, long, double
 - 객체 스트림: 기본 스트림 이외(Stream<String>, Stream<Member>, ...)
- ✓ **IntStream은 [1, 2, 3, ...] 기본형이 들어있는 스트림**
- ✓ **Stream<Integer>는 [new Integer(1), new Integer(2), ...] 참조형이 들어있는 스트림**



데이터 타입별 스트림을 만들어써야함.
할 수 있는 처리 작업이 달라짐.
객체스트림은 바로 평균을 구할 수 없음.

✔ 스트림 인터페이스

리턴 타입	메소드(매개변수)	소스
Stream<T>	java.util.Collection.stream() java.util.Collection.parallelStream()	List 컬렉션 Set 컬렉션
Stream<T> IntStream LongStream DoubleStream	Arrays.stream(T[]), Stream.of(T[]) Arrays.stream(int[]), IntStream.of(int[]) Arrays.stream(long[]), LongStream.of(long[]) Arrays.stream(double[]), DoubleStream.of(double[])	배열
IntStream	IntStream.range(int, int) IntStream.rangeClosed(int, int)	int 범위
LongStream	LongStream.range(long, long) LongStream.rangeClosed(long, long)	long 범위
Stream<Path>	Files.list(Path)	디렉토리
Stream<String>	Files.lines(Path, Charset)	텍스트 파일
DoubleStream IntStream LongStream	Random.doubles(...) Random.ints() Random.longs()	랜덤 수

1.list, set, map등
을 stream으로!

2.배열을
stream으로!

3.일정 범위를
stream으로!

4.랜덤한 수를
stream으로!

✓ 컬렉션으로부터 스트림 얻기

- `java.util.Collection` 인터페이스는 `stream()`과 `parallelStream()` 메소드를 가지고 있어 자식 인터페이스인 `List`와 `Set` 인터페이스를 구현한 모든 컬렉션에서 객체 스트림을 얻을 수 있음

리턴 타입	메소드(매개변수)	소스
Stream<T>	<code>java.util.Collection.stream()</code>	List 컬렉션
	<code>java.util.Collection.parallelStream()</code>	Set 컬렉션

 Product.java

```
package ch17.sec04.exam01;

@AllArgsConstructor
@Data
public class Product {
    private int pno;
    private String name;
    private String company;
    private int price;
}
```

 StreamExample.java

```
package ch17.sec04.exam01;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Stream;

public class StreamExample {
    public static void main(String[] args) {
        //List 컬렉션 생성
        List<Product> list = new ArrayList<>();
        for(int i=1; i<=5; i++) {
            Product product = new Product(i, "상품"+i, "멋진회사", (int)(10000*Math.random()));
            list.add(product);
        }

        //객체 스트림 얻기
        Stream<Product> stream = list.stream();
        stream.forEach(p -> System.out.println(p));
    }
}
```

```
{pno:1, name:상품1, company:멋진회사, price:6188}
{pno:2, name:상품2, company:멋진회사, price:2510}
{pno:3, name:상품3, company:멋진회사, price:9932}
{pno:4, name:상품4, company:멋진회사, price:4317}
{pno:5, name:상품5, company:멋진회사, price:170}
```

✓ 배열로부터 스트림 얻기

- 다양한 타입의 배열을 스트림객체로 만들어야 스트림에서 제공하는 함수를 사용할 수 있다.

리턴 타입	메소드(매개변수)	소스
Stream<T>	Arrays.stream(T[]), Stream.of(T[])	배열
IntStream	Arrays.stream(int[]), IntStream.of(int[])	
LongStream	Arrays.stream(long[]), LongStream.of(long[])	
DoubleStream	Arrays.stream(double[]), DoubleStream.of(double[])	

StreamExample.java

```
public class StreamExample {
    public static void main(String[] args) {
        String[] strArray = { "홍길동", "신용권", "김미나" };

        //int, long, double값 이외에는 “객체 스트림”으로 생성됨.
        Stream<String> strStream = Arrays.stream(strArray);
        strStream.forEach(item -> System.out.print(item + ", "));
        System.out.println();

        //int, long, double은 “기본 스트림”으로 생성됨.
        int[] intArray = { 1, 2, 3, 4, 5 };
        IntStream intStream = Arrays.stream(intArray);
        intStream.forEach(item -> System.out.print(item + ", "));
        System.out.println();
    }
}
```

객체스트림

```
void forEach(Consumer<? super T> action);
```

```
@FunctionalInterface
public interface Consumer<T> {
```

Performs this operation on the given

Params: `t` - the input argument

```
void accept(T t);
```

Accept()메서드를 재정의한 람다함수

기본스트림

홍길동,신용권,김미나,
1,2,3,4,5,

✔ 숫자 범위로부터 스트림 얻기

- `IntStream` 또는 `LongStream`의 정적 메소드인 `range()`와 `rangeClosed()` 메소드로 특정 범위의 정수 스트림을 얻을 수 있음
- 범위를 설정하여 스트림을 만들 수 있음.

리턴 타입	메소드(매개변수)	소스
<code>IntStream</code>	<code>IntStream.range(int, int)</code> <code>IntStream.rangeClosed(int, int)</code>	int 범위
<code>LongStream</code>	<code>LongStream.range(long, long)</code> <code>LongStream.rangeClosed(long, long)</code>	long 범위

 StreamExample.java

```
package ch17.sec04.exam03;

import java.util.stream.IntStream;

public class StreamExample {
    public static int sum;

    public static void main(String[] args) {
        IntStream stream = IntStream.rangeClosed(1, 100);
        stream.forEach(a -> sum += a);
        System.out.println("총합: " + sum);
    }
}
```

총합: 5050

✓ 파일로부터 스트림 얻기

- `java.nio.file.Files`의 `lines()` 메소드로 텍스트 파일의 행 단위 스트림을 얻을 수 있음

리턴 타입	메소드(매개변수)	소스
<code>Stream<Path></code>	<code>Files.list(Path)</code>	디렉토리
<code>Stream<String></code>	<code>Files.lines(Path, Charset)</code>	텍스트 파일

 data.txt

```
{"pno":1, "name":"상품1", "company":"멋진회사", "price":1558}  
{"pno":2, "name":"상품2", "company":"멋진회사", "price":4671}  
{"pno":3, "name":"상품3", "company":"멋진회사", "price":470}  
{"pno":4, "name":"상품4", "company":"멋진회사", "price":9584}  
{"pno":5, "name":"상품5", "company":"멋진회사", "price":6868}
```

 StreamExample.java

```
package ch17.sec04.exam04;

import java.nio.charset.Charset;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.stream.Stream;

public class StreamExample {
    public static void main(String[] args) throws Exception {
        Path path = Paths.get(StreamExample.class.getResource("data.txt").toURI());
        Stream<String> stream = Files.lines(path, Charset.defaultCharset());
        stream.forEach(line -> System.out.println(line) );
        stream.close();
    }
}
```

```
{"pno":1, "name":"상품1", "company":"멋진회사", "price":1558}
{"pno":2, "name":"상품2", "company":"멋진회사", "price":4671}
{"pno":3, "name":"상품3", "company":"멋진회사", "price":470}
{"pno":4, "name":"상품4", "company":"멋진회사", "price":9584}
{"pno":5, "name":"상품5", "company":"멋진회사", "price":6868}
```

5 요소 걸러내기(필터링)

✓ 필터링

- 필터링은 요소를 걸러내는 중간 처리 기능

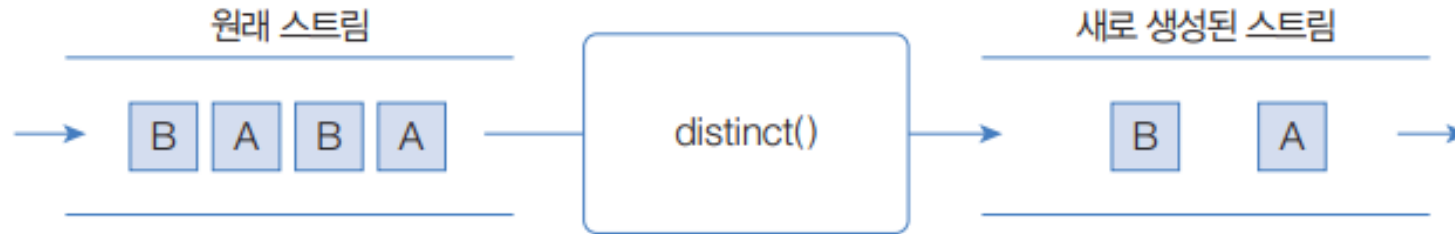
리턴 타입	메소드(매개변수)	설명
Stream IntStream LongStream DoubleStream	distinct()	- 중복 제거
	filter(Predicate<T>) filter(IntPredicate) filter(LongPredicate) filter(DoublePredicate)	- 조건 필터링 - 매개 타입은 요소 타입에 따른 함수형 인터페이스이므로 람다식으로 작성 가능

5 요소 걸러내기(필터링)

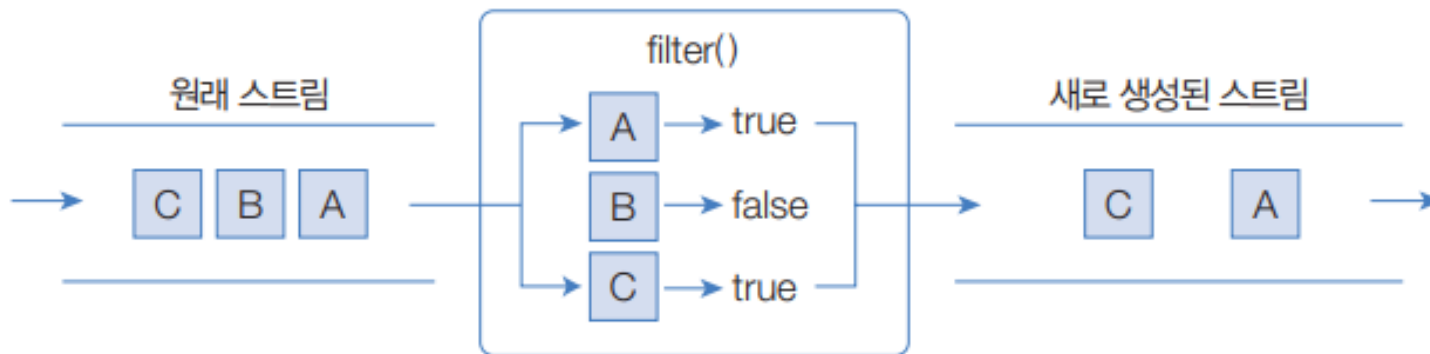
stream객체로 만든
후 할 수 있는 필터링
작업들

✓ 필터링

- `distinct()` 메소드: 요소의 **중복을 제거**



- `filter()` 메소드: 매개값으로 주어진 Predicate(조건식)가 `true`를 리턴하는 요소만 필터링



예) $x > 10$ 이 `true`인 경우에만

5 요소 걸러내기(필터링)

FilteringExample.java

```
package ch17.sec05;

import java.util.ArrayList;
import java.util.List;

public class FilteringExample {
    public static void main(String[] args) {
        //List 컬렉션 생성
        List<String> list = new ArrayList<>();
        list.add("홍길동");
        list.add("신용권");
        list.add("감자바");
        list.add("신용권");
        list.add("신민철");
    }
}
```

5 요소 걸러내기(필터링)

 FilteringExample.java

```
//중복 요소 제거
list.stream()
    .distinct()
    .forEach(n -> System.out.println(n));
System.out.println();
```

중복제거하고 반복문으로 프린트

```
//신으로 시작하는 요소만 필터링
list.stream()
    .filter(n -> n.startsWith("신"))
    .forEach(n -> System.out.println(n));
System.out.println();
```

신으로 시작하는 것만 필터링하여
반복문으로
프린트

```
//중복 요소를 먼저 제거하고, 신으로 시작하는 요소만 필터링
list.stream()
    .distinct()
    .filter(n -> n.startsWith("신"))
    .forEach(n -> System.out.println(n));
}
}
```

중복을 제거한 후,
신으로 시작하는 것만 필터링하여
반복문으로 프린트

홍길동
신용권
감자바
신민철

신용권
신용권
신민철

신용권
신민철

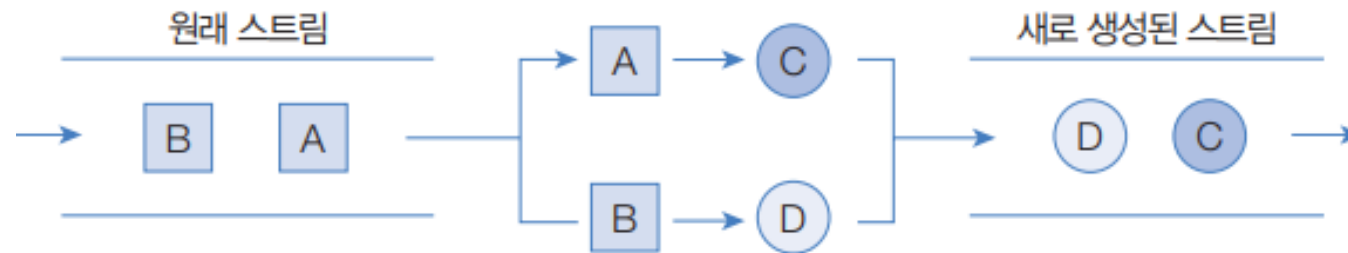
6 요소 변환(매핑)

✓ 매핑

- 스트림의 요소를 다른 요소로 변환하는 중간 처리 기능
- 스트림에 들어있는 요소에 모두 같은 처리를 하는 경우 사용
- 매핑 메소드: `mapXxx()`, `asDoubleStream()`, `asLongStream()`, `boxed()`, `flatMapXxx()` 등

✓ 요소를 다른 요소로 변환

- `mapXxx()` 메소드: 요소를 다른 요소로 변환한 새로운 스트림을 리턴



모든 요소에 동일한 처리한 결과로
새로운 스트림 생성

요소 변환(매핑)

✓ 매핑

○ 주요 메소드

리턴 타입	메소드(매개변수)	요소 → 변환 요소
Stream<R>	map(Function<T, R>)	T → R
IntStream LongStream DoubleStream	mapToInt(ToIntFunction<T>)	T → int
	mapToLong(ToLongFunction<T>)	T → long
	mapToDouble(ToDoubleFunction<T>)	T → double
Stream<U>	mapToObj(IntFunction<U>)	int → U
	mapToObj(LongFunction<U>)	long → U
	mapToObj(DoubleFunction<U>)	double → U
DoubleStream DoubleStream IntStream LongStream	mapToDouble(IntToDoubleFunction)	int → double
	mapToDouble(LongToDoubleFunction)	long → double
	mapToInt(DoubleToIntFunction)	double → int
	mapToLong(DoubleToLongFunction)	double → long

각 객체에 들어있는 요소들을 모두 특정 기능을 동일하게 처리해서 기본형 스트림으로 변경해라라는 의미

기본형 스트림을 다른 기본형 스트림으로 변경해라라는 의미

 Student.java

```
package ch17.sec06.exam01;

public class Student {
    private String name;
    private int score;

    public Student(String name, int score) {
        this.name = name;
        this.score = score;
    }

    public String getName() { return name; }
    public int getScore() { return score; }
}
```

 MapExample.java

```
package ch17.sec06.exam01;

import java.util.ArrayList;
import java.util.List;

public class MapExample {
    public static void main(String[] args) {
        //List 컬렉션 생성
        List<Student> studentList = new ArrayList<>();
        studentList.add(new Student("홍길동", 85));
        studentList.add(new Student("홍길동", 92));
        studentList.add(new Student("홍길동", 87));

        //Student를 score 스트림으로 변환
        studentList.stream()
            .mapToInt(s -> s.getScore())
            .forEach(score -> System.out.println(score));
    }
}
```

```
85
92
87
```

6 요소 변환(매핑)

✓ 매핑

- 기본 타입 간의 변환이거나 기본 타입 요소를 래퍼(Wrapper) 객체 요소로 변환하려면 간편화 메소드를 사용할 수 있음

리턴 타입	메소드(매개변수)	설명
LongStream	asLongStream()	int → long
DoubleStream	asDoubleStream()	int → double long → double
Stream<Integer> Stream<Long> Stream<Double>	boxed()	int → Integer long → Long double → Double

 MapExample.java

```
package ch17.sec06.exam02;

import java.util.Arrays;
import java.util.stream.IntStream;

public class MapExample {
    public static void main(String[] args) {
        int[] intArray = { 1, 2, 3, 4, 5};

        IntStream intStream = Arrays.stream(intArray);
        intStream
            .asDoubleStream()
            .forEach(d -> System.out.println(d));

        System.out.println();

        intStream = Arrays.stream(intArray);
        intStream
            .boxed()
            .forEach(obj -> System.out.println(obj.intValue()));
    }
}
```

```
1.0
2.0
3.0
4.0
5.0

1
2
3
4
5
```


7 요소 정렬

✓ 정렬

- 요소를 오름차순 또는 내림차순으로 정렬하는 중간 처리 기능

리턴 타입	메소드(매개변수)	설명
Stream<T>	sorted()	Comparable 요소를 정렬한 새로운 스트림 생성
Stream<T>	sorted(Comparator<T>)	요소를 Comparator에 따라 정렬한 새 스트림 생성
DoubleStream	sorted()	double 요소를 올림차순으로 정렬
IntStream	sorted()	int 요소를 올림차순으로 정렬
LongStream	sorted()	long 요소를 올림차순으로 정렬

✓ 매칭

- 요소들이 특정 조건에 만족하는지 여부를 조사하는 최종 처리 기능
- `allMatch()`, `anyMatch()`, `noneMatch()` 메소드는 매개값으로 주어진 Predicate가 리턴하는 값에 따라 `true` 또는 `false`를 리턴

리턴 타입	메소드(매개변수)	조사 내용
boolean	<code>allMatch(Predicate<T> predicate)</code> <code>allMatch(IntPredicate predicate)</code> <code>allMatch(LongPredicate predicate)</code> <code>allMatch(DoublePredicate predicate)</code>	모든 요소가 만족하는지 여부
boolean	<code>anyMatch(Predicate<T> predicate)</code> <code>anyMatch(IntPredicate predicate)</code> <code>anyMatch(LongPredicate predicate)</code> <code>anyMatch(DoublePredicate predicate)</code>	최소한 하나의 요소가 만족하는지 여부
boolean	<code>noneMatch(Predicate<T> predicate)</code> <code>noneMatch(IntPredicate predicate)</code> <code>noneMatch(LongPredicate predicate)</code> <code>noneMatch(DoublePredicate predicate)</code>	모든 요소가 만족하지 않는지 여부

 MatchingExample.java

```
package ch17.sec09;

import java.util.Arrays;

public class MatchingExample {
    public static void main(String[] args) {
        int[] intArr = { 2, 4, 6 };

        boolean result = Arrays.stream(intArr)
            .allMatch(a -> a%2==0);
        System.out.println("모두 2의 배수인가? " + result);

        result = Arrays.stream(intArr)
            .anyMatch(a -> a%3==0);
        System.out.println("하나라도 3의 배수가 있는가? " + result);

        result = Arrays.stream(intArr)
            .noneMatch(a -> a%3==0);
        System.out.println("3의 배수가 없는가? " + result);
    }
}
```

```
모두 2의 배수인가? true
하나라도 3의 배수가 있는가? true
3의 배수가 없는가? false
```

9 요소 기본 집계

✓ 집계

- 최종 처리 기능으로 요소들을 처리해서 카운팅, 합계, 평균값, 최대값, 최소값 등 하나의 값으로 산출하는 것

✓ 스트림이 제공하는 기본 집계

- 스트림은 카운팅, 최대, 최소, 평균, 합계 등을 처리하는 다음과 같은 최종 처리 메소드를 제공

리턴 타입	메소드(매개변수)	설명
long	count()	요소 개수
OptionalXXX	findFirst()	첫 번째 요소
Optional<T> OptionalXXX	max(Comparator<T>) max()	최대 요소
Optional<T> OptionalXXX	min(Comparator<T>) min()	최소 요소
OptionalDouble	average()	요소 평균
int, long, double	sum()	요소 총합

결과가 없을 수
있음.

AggregateExample.java

```
package ch17.sec10;

import java.util.Arrays;

public class AggregateExample {
    public static void main(String[] args) {
        //정수 배열
        int[] arr = {1, 2, 3, 4, 5};

        //카운팅
        long count = Arrays.stream(arr)
            .filter(n -> n%2==0)
            .count();
        System.out.println("2의 배수 개수: " + count);

        //총합
        long sum = Arrays.stream(arr)
            .filter(n -> n%2==0)
            .sum();
        System.out.println("2의 배수의 합: " + sum);
    }
}
```

2의 배수 개수: 2
2의 배수의 합: 6

AggregateExample.java

```
//평균
double avg = Arrays.stream(arr)
    .filter(n -> n%2==0)
    .average()
    .getAsDouble();
System.out.println("2의 배수의 평균: " + avg);

//최대값
int max = Arrays.stream(arr)
    .filter(n -> n%2==0)
    .max()
    .getAsInt();
System.out.println("최대값: " + max);

//최소값
int min = Arrays.stream(arr)
    .filter(n -> n%2==0)
    .min()
    .getAsInt();
System.out.println("최소값: " + min);
```

2의 배수의 평균: 3.0
최대값: 4
최소값: 2

 AggregateExample.java

```
//첫 번째 요소  
int first = Arrays.stream(arr)  
    .filter(n -> n%3==0)  
    .findFirst()  
    .getAsInt();  
System.out.println("첫 번째 3의 배수: " + first);  
}  
}
```

첫 번째 3의 배수: 3

문제 설명

머쓱이는 학교에서 키 순으로 줄을 설 때 몇 번째로 서야 하는지 궁금해졌습니다. 머쓱이네 반 친구들의 키가 담긴 정수 배열 `array` 와 머쓱이의 키 `height` 가 매개변수로 주어질 때, 머쓱이보다 키 큰 사람 수를 return 하도록 solution 함수를 완성해보세요.

제한사항

- $1 \leq \text{array}$ 의 길이 ≤ 100
- $1 \leq \text{height} \leq 200$
- $1 \leq \text{array}$ 의 원소 ≤ 200

입출력 예

array	height	result
[149, 180, 192, 170]	167	3
[180, 120, 140]	190	0

```
package stream2;
```

```
import java.util.Arrays;
```

```
public class HeightTest {  
    public static void main(String[] args) {  
        int[] array = { 149, 180, 192, 170 };  
        int height = 167;  
  
        long result = Arrays.stream(array)  
            .filter(value -> value > height)  
            .count(); //long  
        System.out.println(result);  
    }  
}
```

3

문제 설명

문자열 배열 `strlist` 가 매개변수로 주어집니다. `strlist` 각 원소의 길이를 담은 배열을 retrun 하도록 solution 함수를 완성해주세요.

제한사항

- $1 \leq \text{strlist 원소의 길이} \leq 100$
- `strlist` 는 알파벳 소문자, 대문자, 특수문자로 구성되어 있습니다.

입출력 예

strlist	result
["We", "are", "the", "world!"]	[2, 3, 3, 6]
["I", "Love", "Programmers."]	[1, 4, 12]

```
package stream2;
```

```
import java.util.Arrays;
```

```
public class StrList {
    public static void main(String[] args) {
```

```
        String[] strList = {"We", "are", "the", "world!"};
```

```
        int[] result = Arrays.stream(strList)
```

```
            .mapToInt(String::length)
```

```
            .toArray(); //int[]로 변환
```

```
        System.out.println(Arrays.toString(result));
```

```
    }
```

```
}
```

```
[2, 3, 3, 6]
```

배열의 유사도

문제 설명

두 배열이 얼마나 유사한지 확인해보려고 합니다. 문자열 배열 `s1` 과 `s2` 가 주어질 때 같은 원소의 개수를 return하도록 solution 함수를 완성해주세요.

제한사항

- $1 \leq s1, s2$ 의 길이 ≤ 100
- $1 \leq s1, s2$ 의 원소의 길이 ≤ 10
- `s1` 과 `s2` 의 원소는 알파벳 소문자로만 이루어져 있습니다
- `s1` 과 `s2` 는 각각 중복된 원소를 갖지 않습니다.

입출력 예

s1	s2	result
["a", "b", "c"]	["com", "b", "d", "p", "c"]	2
["n", "omg"]	["m", "dot"]	0

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

public class ArraySame {
    public static void main(String[] args) {
        String[] s1 = {"a", "b", "c"};
        String[] s2 = {"com", "b", "d", "p", "c"};

        //s1의 중복을 제거한 목록을 구한다.
        Set<String> set = new HashSet<>();
        for (String s : s1) {
            set.add(s);
        }

        //s2를 stream을 만들고
        //set에 넣은 것을 포함하고 있는지 filtering한 후
        //카운트
        long result = Arrays.stream(s2)
            .filter(set::contains)
            .count();
        System.out.println(result);
    }
}
```

2

문제 설명

정수 `n` 이 매개변수로 주어질 때 `n` 의 각 자리 숫자의 합을 return하도록 solution 함수를 완성해주세요

제한사항

- $0 \leq n \leq 1,000,000$

입출력 예

n	result
1234	10
930211	16

```
package stream2;
```

```
import java.util.Arrays;
```

```
public class SumTest {
```

```
    public static void main(String[] args) {
```

```
        int n = 1234;
```

```
        //1. 문자열로 만들어 split(분리)하면 String[]이 만들어짐.
```

```
        //{ "1", "2", "3", "4" }
```

```
        //2. stream으로 만들어 하나씩 꺼내 숫자로 변환 후
```

```
        //3. sum()함수로 더함.
```

```
        String[] split = String.valueOf(n).split("");
```

```
        long result = Arrays.stream(split)
```

```
            .mapToInt(Integer::parseInt)
```

```
            .sum();
```

```
        System.out.println(result);
```

```
    }
```

```
}
```

10

average()함수는 결과값 확인 필요

✓ Optional 클래스

- Optional, OptionalDouble, OptionalInt, OptionalLong 클래스는 단순히 집계값만 저장하는 것이 아니라, 집계값이 없으면 디폴트 값을 설정하거나 집계값을 처리하는 Consumer를 등록

리턴 타입	메소드(매개변수)	설명
boolean	isPresent()	집계값이 있는지 여부
T double int long	orElse(T) orElse(double) orElse(int) orElse(long)	집계값이 없을 경우 디폴트 값 설정
void	ifPresent(Consumer) ifPresent(DoubleConsumer) ifPresent(IntConsumer) ifPresent(LongConsumer)	집계값이 있을 경우 Consumer에서 처리

9 average()함수는 결과값 확인 필요

✓ Optional 클래스

- isPresent() 메소드가 true를 리턴할 때만 집계값 얻기

```
OptionalDouble optional = stream
    .average();
if(optional.isPresent()) {
    System.out.println("평균: " + optional.getAsDouble());
} else {
    System.out.println("평균: 0.0");
}
```



방법1

9 요소 기본 집계

✓ Optional 클래스

- `orElse()` 메소드로 집계값이 없을 경우 대비해서 디폴트 값 정해놓기

```
double avg = stream
    .average()
    .orElse(0.0);
System.out.println("평균: " + avg);
```

방법2

- `ifPresent()` 메소드로 집계값이 있을 경우에만 동작하는 Consumer 랴다식 제공하기

```
stream
    .average()
    .ifPresent(a -> System.out.println("평균: " + a));
```

방법3

```
package stream2;

import java.util.Arrays;
import java.util.OptionalDouble;

public class AverageTest {
    public static void main(String[] args) {
        int[] n = {1, 2, 3, 4, 5};
        int[] n2 = new int[0];

        OptionalDouble result = Arrays.stream(n2)
            .average(); //OptionalDouble[0.0]

        System.out.println(result); //OptionalDouble[0.0]

        //average()함수는 결과값이 없을 수도 있음.
        if (result.isPresent()) {
            System.out.println(result.getAsDouble()); //0.0
        } else {
            System.out.println("평균 : 0.0");
        }
    }
}
```

AverageTest.java

```
package stream2;

import java.util.Arrays;
import java.util.OptionalDouble;

public class AverageTest {
    public static void main(String[] args) {
        int[] n = {1, 2, 3, 4, 5};
        int[] n2 = new int[0];

        //방법1 -> 있는지 없는지 체크해서 다르게 처리
        OptionalDouble result = Arrays.stream(n2)
            .average(); //OptionalDouble[0.0]

        System.out.println(result); //OptionalDouble[0.0]

        //average()함수는 결과값이 없을 수도 있음.
        if (result.isPresent()) {
            //OptionalDouble에서 double값을 꺼내야한다.
            System.out.println("평균: " + result.getAsDouble()); //0.0
        } else {
            System.out.println("평균 : 0.0");
        }
    }
}
```


AverageTest.java

//방법2 -> 없는 경우 기본값을 설정

```
double result2 = Arrays.stream(n2)
    .average()
    .orElse(0.0); //OptionalDouble에 있는 orElse()메서드 호출
System.out.println("평균: " + result2);
```

//방법3 -> 있는 경우만 프린트

```
Arrays.stream(n2)
    .average()
    .ifPresent(x -> System.out.println("평균: " + x)); //OptionalDouble에 있는 ifPresend()메서드 호출
}
}
```

 OptionalExample.java

```
package ch17.sec10;

import java.util.ArrayList;
import java.util.List;
import java.util.OptionalDouble;

public class OptionalExample {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();

        /**//예외 발생(java.util.NoSuchElementException)
        double avg = list.stream()
            .mapToInt(Integer :: intValue)
            .average()
            .getAsDouble();
        */
    }
}
```

OptionalExample.java

//방법1

```
OptionalDouble optional = list.stream()
    .mapToInt(Integer :: intValue)
    .average();
if(optional.isPresent()) {
    System.out.println("방법1_평균: " + optional.getAsDouble());
} else {
    System.out.println("방법1_평균: 0.0");
}
```

//방법2

```
double avg = list.stream()
    .mapToInt(Integer :: intValue)
    .average()
    .orElse(0.0);
System.out.println("방법2_평균: " + avg);
```

//방법3

```
list.stream()
    .mapToInt(Integer :: intValue)
    .average()
    .ifPresent(a -> System.out.println("방법3_평균: " + a));
}
```

방법1_평균: 0.0
방법2_평균: 0.0